```
suspend fun initializeData(recipes: List<Recipe>) {
        val existing = recipeCollection.limit(1).get().await()
        if (existing.isEmpty) {
            firestore.runBatch { batch ->
                recipes.forEach { recipe ->
                    Log.d("DEBUG", "$recipe")
                    val docRef =
recipeCollection.document(recipe.recipeId)
                    batch.set(docRef, recipe)
                }
            }.await()
        }
    }
```

every recipe is like this:


```
{
    "recipeId": "101",
    "originalRecipeId": null,
    "authorId": "101",
    "title": "Spicy Arrabbiata Pasta",
    "description": "Classic Italian pasta with a spicy tomato
sauce.",
    "photoURL": "https://images.unsplash.com/photo-
1608756687911-aa1599ab3bd9",
    "preparationTime": 20,
    "servings": 2,
    "rating": 4.7,
    "numberOfRatings": 80,
    "difficulty": "EASY",
    "cost": "CHEAP",
    "calories": 520,
    "ingredients": [
     { "recipeIngredientId": "ri8", "ingredient": {
"ingredientId": "8", "name": "Pasta",
"unitOfMeasurementToCaloryPerUnit": {"g": 131},
"photoURL": null, "isUserCreated": false, "userId": null },
"quantity": 200.0, "customUnitOfMeasurement": "g" },
     { "recipeIngredientId": "ri9", "ingredient": {
"ingredientId": "9", "name": "Tomato Sauce",
"unitOfMeasurementToCaloryPerUnit": {"g": 29},
```

"photoURL": null, "isUserCreated": false, "userId": null },
"quantity": 150.0, "customUnitOfMeasurement": "g" }
],
"steps": [
{ "stepId": "s5", "photoURL": null, "description": "Boil pasta until al dente.", "order": 1 },
{ "stepId": "s6", "photoURL": null, "description": "Heat tomato sauce with chili flakes.", "order": 2 },
{ "stepId": "s7", "photoURL": null, "description": "Combine pasta and sauce.", "order": 3 }
],
"tags": [
{ "tagId": "2", "type": "Cuisine Type", "name": "Italian" },
{ "tagId": "6", "type": "Other", "name": "Spicy" }
],
"reviews": []
},

as you can see the recipe itself can have a photo, but also each step can have a photo, i need when i init to upload the photos to firebase, this is what i do when i upload a new recipe:

```
override suspend fun addOrOverrideRecipe(recipe: Recipe) = withContext(Dispatchers.IO) {
    Log.d("CLONED FROM", "${recipe.originalRecipeId}")
    var uploadedRecipeImageUrl: String? = null
    val uploadedStepImages = mutableListOf<Pair<String, String>>()
    try {

        val currentRecipe = recipeCollection.document(recipe.recipeId).get().await().toObject(Recipe::class.java)
        val photoUrl: String = when {
            recipe.photoURL.startsWith("content://") -> {
                uploadRecipeImage(recipe.recipeId, recipe.photoURL).also {
                    uploadedRecipeImageUrl = it
                }
            }

            recipe.photoURL.startsWith("http") &&
```

```
currentRecipe == null  -> {
                val oldRef =
storage.getReferenceFromUrl(recipe.photoURL)
                val newRef = storage.reference.child(

"recipes/${recipe.recipeId}/recipe_${System.currentTimeMil
lis()}.jpg"
                )

                val stream = oldRef.stream.await().stream
                newRef.putStream(stream).await()

                newRef.downloadUrl.await().toString()
            }

            else -> recipe.photoURL
        }

        val updatedSteps = recipe.steps.map { step ->
            val updatedPhotoURL = step.photoURL?.let { url
->
                when {
                    url.startsWith("content://") -> {
                        uploadStepImage(recipe.recipeId,
step.stepId, url).also { uploadedUrl ->
                            uploadedStepImages.add(step.stepId
to uploadedUrl)
                        }
                    }

                    url.startsWith("http") && currentRecipe ==
null -> {
                        val oldRef =
storage.getReferenceFromUrl(url)
                        val newRef = storage.reference.child(

"recipes/${recipe.recipeId}/steps/${step.stepId}.jpg"
                        )

                        val stream = oldRef.stream.await().stream
                        newRef.putStream(stream).await()

                        newRef.downloadUrl.await().toString()
```

```
                    }

                    else -> url
                }
            }

            step.copy(photoURL = updatedPhotoURL)
        }

        val updatedRecipe = recipe.copy(
            photoURL = photoUrl,
            steps = updatedSteps,
            originalRecipeId =
currentRecipe?.originalRecipeId
        )

recipeCollection.document(updatedRecipe.recipeId).set(up
datedRecipe).await()

        // clean up old photo if photo has changed
        if (currentRecipe?.photoURL != null &&
            currentRecipe.photoURL !=
updatedRecipe.photoURL
        ) {
            deleteRecipeImage(currentRecipe.photoURL)
        }

        // clean up old steps photos if they have changed
        currentRecipe?.steps?.forEach { oldStep ->
            val newStep = updatedRecipe.steps.find {
it.stepId == oldStep.stepId }
            val oldUrl = oldStep.photoURL
            val newUrl = newStep?.photoURL

            if (!oldUrl.isNullOrBlank() && oldUrl != newUrl) {
                deleteRecipeImage(oldUrl)
            }
        }



        Log.d("FirebaseRepository", "Success
addOrOverrideRecipe")
```

```
        } catch (e: Exception) {
            Log.e("FirebaseRepository", "Error
addOrOverrideRecipe", e)
            try {
                if (uploadedRecipeImageUrl != null)
deleteRecipeImage(uploadedRecipeImageUrl)

                uploadedStepImages.forEach { (_, url) ->
                    deleteRecipeImage(url)
                }
            } catch (e: Exception) {
                Log.e("FirebaseRepository", "Cleanup failed", e)
            }
            //throw e
        }
    }

    suspend fun deleteRecipeImage(photoUrl: String?) {
        if (photoUrl.isNullOrBlank()) return
        try {
            val imageRef =
storage.getReferenceFromUrl(photoUrl)
            imageRef.delete().await()
        } catch (e: Exception) {
            Log.e("FirebaseStorage", "Failed to delete image",
e)
            //throw e
        }
    }

    suspend fun uploadRecipeImage(recipeId: String, uri:
String): String {
        val imageRef =
storageRef.child("recipes/$recipeId/recipe${System.current
TimeMillis()}.jpg")
        val fileUri = uri.toUri()
        imageRef.putFile(fileUri).await()
        return imageRef.downloadUrl.await().toString()
    }

    suspend fun uploadStepImage(recipeId: String, stepId:
String, uri: String): String {
        val imageRef =
```

```
storageRef.child("recipes/$recipeId/steps/$stepId.jpg")
        val fileUri = uri.toUri()
        imageRef.putFile(fileUri).await()
        return imageRef.downloadUrl.await().toString()
    }
```

the images i have right now are all from the internet, so i need to upload them to firebase storage and use those links

Mostra meno

You should preprocess the recipes before batching them into Firestore:

1. Download/copy every external image (`http...`) into Firebase Storage
2. Replace the URLs in the `Recipe` object with the Firebase Storage download URLs
3. Save the updated recipe into Firestore

Right now your `initializeData()` directly writes the JSON URLs to Firestore without migrating the images.

You already have almost all the logic inside `addOrOverrideRecipe()` .
The cleanest solution is to extract the "migrate/upload recipe images" logic into a reusable function.

Example:

Kotlin

```kotlin
private suspend fun migrateRecipeImages(recipe: Recipe): Recipe {
    val uploadedRecipePhoto = when {
        recipe.photoURL.startsWith("http") -> {
            val oldRef = storage.getReferenceFromUrl(recipe.photoURL)

            val newRef = storage.reference.child(
                "recipes/${recipe.recipeId}/recipe_${System.currentTimeMillis()}.jpg"
            )

            val stream = oldRef.stream.await().stream
            newRef.putStream(stream).await()

            newRef.downloadUrl.await().toString()
        }

        else -> recipe.photoURL
    }
```

```
    val updatedSteps = recipe.steps.map { step ->
        val updatedStepPhoto = step.photoURL?.let { url ->
            when {
                url.startsWith("http") -> {
                    val oldRef = storage.getReferenceFromUrl(url)

                    val newRef = storage.reference.child(
                        "recipes/${recipe.recipeId}/steps/${step.stepId}.jpg"
                    )

                    val stream = oldRef.stream.await().stream
                    newRef.putStream(stream).await()

                    newRef.downloadUrl.await().toString()
                }

                else -> url
            }
        }

        step.copy(photoURL = updatedStepPhoto)
    }

    return recipe.copy(
        photoURL = uploadedRecipePhoto,
        steps = updatedSteps
    )
}
```

Then your initialization becomes:

Kotlin

```
suspend fun initializeData(recipes: List<Recipe>) {
    val existing = recipeCollection.limit(1).get().await()

    if (existing.isEmpty) {

        val migratedRecipes = recipes.map { recipe ->
            migrateRecipeImages(recipe)
        }

        firestore.runBatch { batch ->
            migratedRecipes.forEach { recipe ->
                Log.d("DEBUG", "$recipe")

                val docRef = recipeCollection.document(recipe.recipeId)
                batch.set(docRef, recipe)
            }
        }.await()
```

```
    }
}
```

A few important notes:

- `storage.getReferenceFromUrl(url)` works only if the URL already points to Firebase Storage.
- Your current URLs are Unsplash URLs, so `getReferenceFromUrl()` will fail.

For external internet URLs you must actually download the image bytes yourself.

So replace:

Kotlin

```
val oldRef = storage.getReferenceFromUrl(url)
val stream = oldRef.stream.await().stream
newRef.putStream(stream).await()
```

with something like:

Kotlin

```
private suspend fun uploadExternalImageToFirebase(
    imageUrl: String,
    storagePath: String
): String = withContext(Dispatchers.IO) {

    val url = java.net.URL(imageUrl)
    val connection = url.openConnection()
    connection.connect()

    val inputStream = connection.getInputStream()

    val ref = storage.reference.child(storagePath)

    ref.putStream(inputStream).await()

    inputStream.close()

    ref.downloadUrl.await().toString()
}
```

Then:

Kotlin

```
val uploadedRecipePhoto = when {
    recipe.photoURL.startsWith("http") -> {
        uploadExternalImageToFirebase(
```

```
                recipe.photoURL,
                "recipes/${recipe.recipeId}/recipe_${System.currentTimeMillis()}.jpg"
            )
        }

        else -> recipe.photoURL
    }
```

And similarly for steps:

Kotlin

```
uploadExternalImageToFirebase(
    url,
    "recipes/${recipe.recipeId}/steps/${step.stepId}.jpg"
)
```

This is the correct approach for migrating external CDN images (Unsplash, etc.) into Firebase Storage.

no, the url in the recipe is not aa url that points to firebase storage, it is an external link, just an image i found online like: https://images.unsplash.com/photo-1608756687911-aa1599ab3bd9

Exactly — that's why `storage.getReferenceFromUrl()` will not work.

That method only works for URLs already inside Firebase Storage, like:

```
https://firebasestorage.googleapis.com/...
```

Your URLs are external CDN URLs from Unsplash:

```
https://images.unsplash.com/photo-1608756687911-aa1599ab3bd9
```

So you must:

1. Download the image from the internet
2. Upload the bytes/stream to Firebase Storage
3. Save the new Firebase download URL into Firestore

This is the correct implementation:

Kotlin

```kotlin
private suspend fun uploadExternalImageToFirebase(
    imageUrl: String,
    storagePath: String
): String = withContext(Dispatchers.IO) {

    val url = URL(imageUrl)
    val connection = url.openConnection()
    connection.connect()

    val inputStream = connection.getInputStream()

    val storageRef = storage.reference.child(storagePath)

    storageRef.putStream(inputStream).await()

    inputStream.close()

    storageRef.downloadUrl.await().toString()
}
```

Then use it in initialization:

Kotlin

```kotlin
suspend fun initializeData(recipes: List<Recipe>) {

    val existing = recipeCollection.limit(1).get().await()

    if (existing.isEmpty) {

        val updatedRecipes = recipes.map { recipe ->

            val uploadedRecipePhoto = if (recipe.photoURL.startsWith("http")) {
                uploadExternalImageToFirebase(
                    recipe.photoURL,
                    "recipes/${recipe.recipeId}/recipe.jpg"
                )
            } else {
                recipe.photoURL
            }

            val updatedSteps = recipe.steps.map { step ->

                val uploadedStepPhoto = step.photoURL?.let { url ->

                    if (url.startsWith("http")) {
                        uploadExternalImageToFirebase(
                            url,
                            "recipes/${recipe.recipeId}/steps/${step.stepId}.jpg"
```

```
                    )
                } else {
                    url
                }
            }

            step.copy(photoURL = uploadedStepPhoto)
        }

        recipe.copy(
            photoURL = uploadedRecipePhoto,
            steps = updatedSteps
        )
    }

    firestore.runBatch { batch ->

        updatedRecipes.forEach { recipe ->

            val docRef = recipeCollection.document(recipe.recipeId)

            batch.set(docRef, recipe)
        }

    }.await()
  }
}
```

That will migrate all recipe and step images from Unsplash → Firebase Storage automatically during initialization.

```
private suspend fun uploadExternalImageToFirebase(
    imageUrl: String,
    storagePath: String
): String = withContext(Dispatchers.IO) {

    val url = URL(imageUrl)
    val connection = url.openConnection()
    connection.connect()
    val inputStream = connection.getInputStream()

    val storageRef = storage.reference.child(storagePath)
    storageRef.putStream(inputStream).await()
    inputStream.close()
```

```
        storageRef.downloadUrl.await().toString()
    }
```

is there a more kotlin native way to implement the dowload of the image?

Mostra meno

Yes. Using `java.net.URL(...).openConnection()` works, but on Android the more idiomatic and modern approach is to use an HTTP client like:

- OkHttp (most common)
- Ktor client
- Coil internals (not ideal here)

For your use case, `OkHttp` is the cleanest and most reliable.

Example:

Kotlin

```kotlin
private val client = OkHttpClient()

private suspend fun uploadExternalImageToFirebase(
    imageUrl: String,
    storagePath: String
): String = withContext(Dispatchers.IO) {

    val request = Request.Builder()
        .url(imageUrl)
        .build()

    client.newCall(request).execute().use { response ->

        if (!response.isSuccessful) {
            throw IOException("Failed to download image: ${response.code}")
        }

        val inputStream = response.body?.byteStream()
            ?: throw IOException("Empty response body")

        val storageRef = storage.reference.child(storagePath)

        storageRef.putStream(inputStream).await()

        storageRef.downloadUrl.await().toString()
```

```
    }
}
```

Dependency:

Kotlin

```
implementation("com.squareup.okhttp3:okhttp:4.12.0")
```

Why this is better:

- Proper HTTP handling
- Timeouts/retries
- Better TLS/redirect support
- Cleaner resource management with `.use`
- Standard Android ecosystem solution
- Easier to extend later (headers, caching, logging, interceptors)

You could also make it fully stream-safe:

Kotlin

```
private suspend fun uploadExternalImageToFirebase(
    imageUrl: String,
    storagePath: String
): String = withContext(Dispatchers.IO) {

    val request = Request.Builder()
        .url(imageUrl)
        .build()

    client.newCall(request).execute().use { response ->

        if (!response.isSuccessful) {
            error("Download failed: ${response.code}")
        }

        response.body!!.byteStream().use { stream ->

            val storageRef = storage.reference.child(storagePath)

            storageRef.putStream(stream).await()

            storageRef.downloadUrl.await().toString()
        }
    }
}
```

That version guarantees every stream is closed correctly even if upload fails.



Android back button navigation in Kotlin

> i'm writing an android app in kotlin. i have a onNavigateBack function in the ui. how do i connect it to the back button of the phone?

You can connect `onNavigateBack` to the system back button using the `BackHandler` API from Jetpack Compose, or the `OnBackPressedCallback` if you're in a Fragment/Activity.

If you're using Jetpack Compose:

```kotlin
import androidx.activity.compose.BackHandler

@Composable
fun MyScreen(onNavigateBack: () -> Unit) {
    BackHandler {
        onNavigateBack()
    }
    // ... rest of your UI
}
```

`BackHandler` is enabled by default and intercepts the back gesture/button press.

If you want to conditionally intercept it (e.g. only when a dialog is open):

```kotlin
BackHandler(enabled = isDialogOpen) {
    onNavigateBack()
}
```

When `enabled = false`, the system handles the back press normally.

If you're using Fragments (View-based UI):

```kotlin
```

```
requireActivity().onBackPressedDispatcher.addCallback(viewLifecycleOwner) {
    onNavigateBack()
}
```

If you're in an Activity:

kotlin

```
onBackPressedDispatcher.addCallback(this) {
    onNavigateBack()
}
```

The Compose `BackHandler` approach is the most common and idiomatic choice for modern Android development. Just drop it anywhere in your composable and it'll wire up automatically.



appNavigation.kt:

```
bottomBar = {
    if (!isCreateRecipeScreen && !isAuthScreen &&
!isPictureScreen && !isPictureDialog) {
        TasteToShareBottomBar(
            currentDestination = currentDestination,
            onNavigateHome = { navActions.navigateToHomePage()
},
            onNavigateShopping = {
navActions.navigateToShoppingList() },
            onNavigateSaved = {
navActions.navigateToSavedRecipes() },
            onNavigateProfile = { currentUserId?.let {
navActions.navigateToProfile(it) }},
            onCreateRecipe = {
                navActions.navigateToCreateRecipe(null, null)
            }
        )
    }
},
```

NavigationActions.kt:

```
fun navigateToProfile(userId: String) {
    if(SessionManagerFacade.isLoggedIn) {
        navCtrl.navigate(ProfileRoute(userId))
    }
    else navCtrl.navigate(WelcomeRoute)
}
```

i have this because i want that an unlogged user can't see their profile page. at the same time, if they are logged in, this navigation must work for both the own profile and some other user's profile. right now, though, when i'm unlogged and i try to click on the profile, nothing happens instead of being redirected to the login screen. how do i solve it?

Mostra meno

The issue is that `currentUserId` is `null` when the user is not logged in, so the `let` block never executes and `navigateToProfile` is never called at all.

You need to separate the two cases: navigate to profile with a user ID when logged in, or navigate to welcome/login when not. The cleanest fix is to handle the unauthenticated case directly in the bottom bar's `onNavigateProfile` lambda:

kotlin

```kotlin
onNavigateProfile = {
    if (currentUserId != null) {
        navActions.navigateToProfile(currentUserId)
    } else {
        navActions.navigateToWelcome() // or navigateToLogin()
    }
}
```

Or if you prefer to keep that logic inside `NavigationActions`, change the signature to accept a nullable ID:

kotlin

```kotlin
fun navigateToProfile(userId: String?) {
    if (SessionManagerFacade.isLoggedIn && userId != null) {
        navCtrl.navigate(ProfileRoute(userId))
    } else {
        navCtrl.navigate(WelcomeRoute)
    }
}
```

Then in `appNavigation.kt` remove the `let`:

kotlin

```kotlin
onNavigateProfile = { navActions.navigateToProfile(currentUserId) }
```

The second approach is slightly cleaner since it keeps all navigation auth-guard logic centralized in `NavigationActions` rather than scattered across call sites.



Google sign-in button icon for Android

> i'm building the login screen of an android app, that has the "sign in with google" button. i already connected my app to perform the login with google. is there some icon or component already included from google or do i have to look on the internet for the G icon to put in the button?

Check frontend design skill

Google provides an **official asset** for this — you don't need to hunt around the internet. Here are your options, from best to most manual:

✅ Option 1: Use the official Google Sign-In button library (recommended)

Google provides a **Credential Manager** integration for Android that includes the properly branded button out of the box. If you're using the modern Auth approach:

kotlin

```
// In your layout XML
<com.google.android.gms.common.SignInButton
    android:id="@+id/sign_in_button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content" />
```

This is part of `play-services-auth` and renders the fully branded button automatically — G logo, correct colors, correct typography — with no extra assets needed.

✅ Option 2: Google's official brand assets (for a custom button)

If you need a **custom-styled button** (to match your app's design), Google provides the official G logo as a downloadable SVG/PNG from their brand guidelines:

👉 **https://developers.google.com/identity/branding-guidelines**

That page includes:

- The official G icon in the correct colors
- Rules on button sizing, padding, and colors
- Light and dark variants

Google requires you to follow these guidelines if you build a custom button.

✅ Option 3: Use Android's built-in vector drawable

Android Studio includes the Google G logo as a built-in vector asset. You can access it via:

1. Right-click `res/drawable` → **New** → **Vector Asset**
2. Click **Clip Art**
3. Search for **"google"**

It may appear depending on your Material icons version. Alternatively, you can paste this directly as `ic_google.xml` in your `res/drawable` folder — it's the standard Google G SVG converted to Android vector format, which you can find freely on GitHub repos like this one.

TL;DR

Approach	Effort	Custom styling
`SignInButton` from GMS	Zero	Limited
Official asset from Google's brand page	Low	Full
Android Studio vector asset	Low	Full

For most apps, **Option 1** is the easiest path. If your design is custom, grab the asset from Google's branding guidelines page — that's the "official" source and keeps you compliant with their brand rules.